

PRACTICAL TASK FIVE

DATABASE & FILESERVER CONNECTIVITY – ADD A NEW RECORD AND VERIFY PK

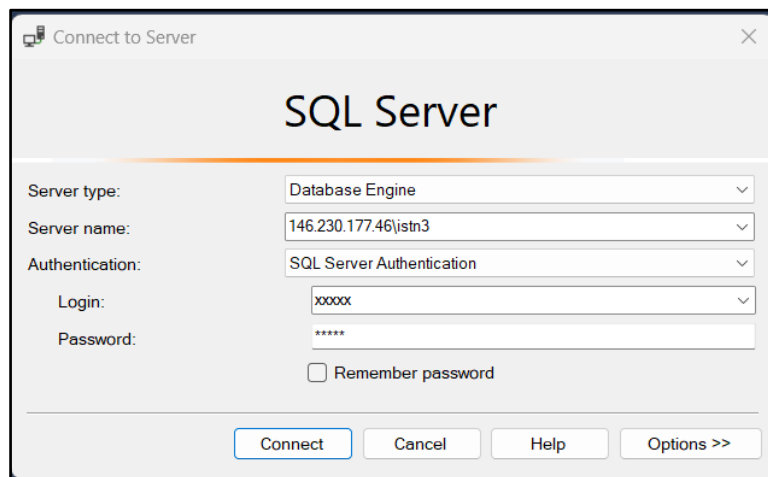
OBJECTIVES

- Establish familiarity with Sql Server development tools
- Test out Sql Server individual accounts on Sql Server
- Create a simple database structures
- Populate database tables with data
- Create a C# application using Visual Studio (VS)
- Establish database connectivity with Sql Server database
- Display data from Sql Server using Windows Forms
- Add a new record and verify the PK of the newly inserted record

PART A

1. Launch Sql Server Management Studio
2. Activate GlobalProtect (University VPN)
3. Establish a connection to the Sql Server
4. Note: The IP address for the SQL Server database is:

146.230.177.46\ISTN3



PART B

This is a continuation of Prac 4

In Prac 4 you would have developed an app to display details of customers from a database table named Customer/Cust. Also, Prac 4 required the creation of a database table named Order. In the current prac both of these tables will be used to enable a Customer to record an order (overall order – not the order details). Prac 5 is a simple addition to the interface where a panel is created with the intention of providing the user with an opportunity to ADD A NEW CUSTOMER.

1. Create the Interface for adding a new Customer. It should look like the illustration below:

The screenshot shows a Windows Forms application titled 'Form1' with two tabs: 'Customer' and 'Order'. The 'Customer' tab is active and contains a form with the following elements:

- A label 'Enter the Customer Surname' above a text box.
- A table with columns: CustId, FirstName, Surname, email, ContactNo. The first row has an asterisk (*) in the CustId column.
- A label 'Customer ID:' above a text box.
- A table with columns: OrderNo, OrderTotal, OrderDate. The first row has an asterisk (*) in the OrderNo column.

An arrow points from the 'Add New Customer' panel on the right to the 'Customer' tab. The 'Add New Customer' panel is a purple box with the following elements:

- A label 'Add New Customer' at the top.
- Two text boxes labeled 'Enter First Name' and 'Enter Surname'.
- Two text boxes labeled 'Enter email address' and 'Enter Contact No'.
- A button labeled 'Add New Customer' at the bottom.

2. To Insert a New Customer, you can create a query on the table Customer table in the app's main dataset; the query is simply an Insert Sql query that takes in the customer details as parameters.

The screenshot shows the 'TableAdapter Query Configuration Wizard' for the 'Cust' table. The 'Choose a Command Type' section is selected, showing the following options:

- ☒ **Use SQL statements**
Specify a SELECT statement to load data.
- ☐ **Create new stored procedure**
Specify a SELECT statement, and the wizard will generate a new stored procedure to select
- ☐ **Use existing stored procedure**
Choose an existing stored procedure.

The screenshot shows the 'TableAdapter Query Configuration Wizard' for the 'Order' table. The 'Choose a Query Type' section is selected, showing the following options:

- ☐ **SELECT which returns rows**
Returns one or many rows or columns.
- ☐ **SELECT which returns a single value**
Returns a single value (for example, Sum, Count, or any other aggregate function).
- ☐ **UPDATE**
Changes existing data in a table.
- ☐ **DELETE**
Removes rows from a table.
- ☒ **INSERT**
Adds a new row to a table.

By default, a Sql Insert Query is automatically generated; it looks like...

What data should the table load?

```
INSERT INTO [dbo].[Cust] ([FirstName], [Surname], [email], [ContactNo], [Rewards]) VALUES (@FirstName,
@Surname, @email, @ContactNo, @Rewards);
SELECT CustId, FirstName, Surname, email, ContactNo, Rewards FROM Cust WHERE (CustId = SCOPE_IDENTITY())
```

This query is designed to Insert a New Customer into the database Customer table; the customer's Firstname, Surname, email, ContactNo and Rewards will be added as parameters when this sql query is implemented. The important thing to understand here is the 2nd part of the query which is optional (and can be deleted). However, the 2nd part makes use of the Scope_Identity() that is used to return to the application, the Primary Key of the newly inserted Customer record.

The Customer Table on the app's dataset now looks like...

Cust	
	CustId
	FirstName
	Surname
	email
	ContactNo
	Rewards
CustTableAdapter	
	Fill,GetData ()
	FillBySurname,GetDataBy (@Surname)
	InsertQueryNewCustomer (@FirstName, @Surname, @email, @ContactNo,...

A new InsertQuery has been added and it will function as a method that will insert a new customer. This method will insert a new customer into the database table and it will return the PK (ID) of the newly inserted customer

3. If you go back to the application code and add in the event handler for the button labelled "Add New Customer"

```
private void btnAddNewCust_Click(object sender, EventArgs e)
{
    int newCustID = (int)taCust.InsertQueryNewCustomer(txtFirst.Text, txtSurname.Text,
txtEmail.Text, txtContact.Text,null);
    taCust.Fill(dsCustOrder.Cust);
    MessageBox.Show("Customer " + txtFirst.Text+ " "+ txtSurname.Text+" has been added with
a Cust ID of: "+newCustID);
    txtFirst.Clear();
    txtSurname.Clear();
    txtEmail.Clear();
    txtContact.Clear();
}
```

This fragment of code inserts the new customer; returns the PK of the newly inserted customer; prints a confirmation message to indicate that the new customer has been inserted into the database table and it prints the ID of the new customer; once the insertion has taken place, all the fields are cleared so that the customer is not inserted into the database for a 2nd and third time. Once this code is working, you can add error handling routines where you check to ensure that all the customer details are provided before the insertion of the customer takes place. This can be done with if...then and try ...catch statements and is left to you as an exercise. Also for the current app, the Rewards field for the Customer will be left blank – this is achieved by using the null value that is recognised in C#

4. One final part – if you would like to view the newly inserted customer inside the Customer database table, the you have to use the Customer table adapter and refill the customer gridview

```
taCust.Fill(dsCustOrder.Cust);
```

5. A sample run.....

The screenshot shows a form titled "Add New Customer" with four input fields: "Enter First Name" (containing "Donald"), "Enter Surname" (containing "Trump"), "Enter email address" (containing "trump@whitehouse.org"), and "Enter Contact No" (containing "777777777"). Below these fields is a button labeled "Add New Customer". To the right, a confirmation message box displays: "Customer Donald Trump has been added with a Cust ID of: 54". An "OK" button is visible at the bottom right of the message box.

And if you take a look at the gridview....

Customer Order					
Enter the Customer Surname					
	CustId	FirstName	Surname	email	ContactNo
	52	Sarah	Brightman	bright@ukzn.ac.za	676786899
	53	Themba	Bavuma	bavuma@gmail.com	3467843
	54	Donald	Trump	trump@whitehouse.org	777777777
*					